

Introduction

Behavioral design patterns focus on the interaction between objects and the communication flow within a system. They help simplify complex logic by promoting clear and efficient communication between different components, reducing dependencies, and enhancing flexibility.

Think of a scenario where a tax calculator visits various types of products – books, electronics, and clothes – and calculates the tax based on the specific type of product. The behavior of how the tax is calculated differs for each product type, but the structure remains flexible and scalable. This is the essence of the **Visitor Pattern**.

In the upcoming sections, we will explore the **Visitor Pattern** in detail, understanding how it allows for the addition of new operations without changing the existing classes.

Visitor Pattern

Formal Definition

The **Visitor Pattern** is a behavioral design pattern that **lets you add new operations to existing class hierarchies without modifying the classes themselves**. This is achieved by moving the logic of the operation into a separate class, known as the "**visitor**".

The main advantage of the **Visitor Pattern** is that it allows you to decouple operations from the objects on which they operate, enabling you to add new operations without changing the classes that contain the objects.

This promotes **Open-Closed Principle (OCP)**, as new functionality can be added without modifying existing code.

Real-Life Analogy

Imagine a **shopping mall** where various shops sell different kinds of products. Each shop (element) has a unique way of applying a discount (operation). Rather than having each shop implement its own method for calculating discounts, we create a **discount visitor** that visits each shop and applies the appropriate discount logic. This way, we can easily add new types of discounts in the future without changing the shop classes.

The Visitor Pattern simplifies complex systems by providing a way to add operations (like discounts) that can be applied to different elements without altering the elements themselves. It decouples operations from the element objects and moves them to separate visitor classes, adhering to the principle of separation of concerns.

Now, let's explore how the Visitor Pattern works in detail through an example.

Understanding the Problem

Let's assume we are building the checkout page of an e-commerce website like Amazon. The checkout process involves various product types such as physical products, gift cards, and digital products. For each product, we need to perform specific operations like calculating shipping costs, discounts, and printing invoices.

Here's a simple way to implement the above requirement:

```
#include <iostream>
```

```
#include <vector>
```

```
#include <memory> // for smart pointers
```

```
using namespace std;
```

```
// Base class for all products
```

```
class Product {
```

```
public:
```

```
    virtual void printInvoice() = 0; // Pure virtual function
```

```
    virtual ~Product() {} // Virtual destructor
```

```
};
```

```
// Class representing a Physical Product
```

```
class PhysicalProduct : public Product {
```

```
public:
```

```
    void printInvoice() override {
```

```
        cout << "Printing invoice for Physical Product..." << endl;
```

```
    }
```

```
    double calculateShippingCost() {
```

```
        cout << "Calculating shipping cost for Physical Product..." << endl;
```

```
        return 10.0; // Example shipping cost
    }
};
```

```
// Class representing a Digital Product
class DigitalProduct : public Product {
public:
    void printInvoice() override {
        cout << "Printing invoice for Digital Product..." << endl;
    }
    // No shipping cost
};
```

```
// Class representing a Gift Card Product
class GiftCard : public Product {
public:
    void printInvoice() override {
        cout << "Printing invoice for Gift Card..." << endl;
    }

    double calculateDiscount() {
```

```

        cout << "Calculating discount for Gift Card..." << endl;

        return 5.0; // Example discount
    }

};

int main() {

    // Create instances of different products using smart pointers
    vector<shared_ptr<Product>> cart = {

        make_shared<PhysicalProduct>(),

        make_shared<DigitalProduct>(),

        make_shared<GiftCard>()

    };


    // Loop through cart and perform actions based on product type
    for (auto& item : cart) {

        if (auto physicalProduct =
dynamic_pointer_cast<PhysicalProduct>(item)) {

            physicalProduct->printInvoice();

            double shippingCost = physicalProduct->calculateShippingCost();

            cout << "Shipping cost: " << shippingCost << "\n" << endl;

        }
    }
}

```

```

        else if (auto digitalProduct =
dynamic_pointer_cast<DigitalProduct>(item)) {

            digitalProduct->printInvoice();

            cout << "No shipping cost for Digital Product." << "\n" << endl;

        }

        else if (auto giftCard = dynamic_pointer_cast<GiftCard>(item)) {

            giftCard->printInvoice();

            double discount = giftCard->calculateDiscount();

            cout << "Discount applied: " << discount << "\n" << endl;

        }

    }

    return 0;

}

```

```
import java.util.*;
```

```
// Class representing a Physical Product
```

```
class PhysicalProduct {
```

```
    // Method to print invoice for physical product
```

```
    void printInvoice() {
```

```
        System.out.println("Printing invoice for Physical Product...");
```

```
}
```

```
// Method to calculate shipping cost for physical product
```

```
double calculateShippingCost() {
```

```
    System.out.println("Calculating shipping cost for Physical Product...");
```

```
    return 10.0; // Example shipping cost
```

```
}
```

```
}
```

```
// Class representing a Digital Product
```

```
class DigitalProduct {
```

```
    // Method to print invoice for digital product
```

```
    void printInvoice() {
```

```
        System.out.println("Printing invoice for Digital Product...");
```

```
    }
```

```
// No shipping cost for digital product
```

```
}
```

```
// Class representing a Gift Card Product
```

```
class GiftCard {
```

```
// Method to print invoice for gift card
```

```
void printInvoice() {
```

```
    System.out.println("Printing invoice for Gift Card...");
```

```
}
```

```
// Method to calculate discount for gift card
```

```
double calculateDiscount() {
```

```
    System.out.println("Calculating discount for Gift Card...");
```

```
    return 5.0; // Example discount
```

```
}
```

```
}
```

```
class Main {
```

```
    public static void main(String[] args) {
```

```
        // Create instances of different products
```

```
        List<Object> cart = Arrays.asList(new PhysicalProduct(), new  
        DigitalProduct(), new GiftCard());
```

```
        // Loop through cart and perform actions based on product type
```

```
        for (Object item : cart) {
```

```
            if (item instanceof PhysicalProduct) {
```

```
                PhysicalProduct physicalProduct = (PhysicalProduct) item;
```



```

        physicalProduct.printInvoice();

        double shippingCost = physicalProduct.calculateShippingCost();

        System.out.println("Shipping cost: " + shippingCost + "\n");
    }

    else if (item instanceof DigitalProduct) {

        DigitalProduct digitalProduct = (DigitalProduct) item;

        digitalProduct.printInvoice();

        System.out.println("No shipping cost for Digital Product." + "\n");
    }

    else if (item instanceof GiftCard) {

        GiftCard giftCard = (GiftCard) item;

        giftCard.printInvoice();

        double discount = giftCard.calculateDiscount();

        System.out.println("Discount applied: " + discount + "\n");
    }

}

}

}

```

Class representing a Physical Product

class PhysicalProduct:

```
# Method to print invoice for physical product
```

```
def printInvoice(self):
```

```
    print("Printing invoice for Physical Product...")
```

```
# Method to calculate shipping cost for physical product
```

```
def calculateShippingCost(self):
```

```
    print("Calculating shipping cost for Physical Product...")
```

```
    return 10.0 # Example shipping cost
```

```
# Class representing a Digital Product
```

```
class DigitalProduct:
```

```
    # Method to print invoice for digital product
```

```
    def printInvoice(self):
```

```
        print("Printing invoice for Digital Product...")
```

```
    # No shipping cost for digital product
```

```
# Class representing a Gift Card Product
```

```
class GiftCard:
```

Method to print invoice for gift card

def printInvoice(self):

print("Printing invoice for Gift Card...")

Method to calculate discount for gift card

def calculateDiscount(self):

print("Calculating discount for Gift Card...")

return 5.0 # Example discount

def main():

Create instances of different products

cart = [PhysicalProduct(), DigitalProduct(), GiftCard()]

Loop through cart and perform actions based on product type

for item in cart:

if isinstance(item, PhysicalProduct):

item.printInvoice()

shippingCost = item.calculateShippingCost()

print(f"Shipping cost: {shippingCost} \n")

elif isinstance(item, DigitalProduct):

```

        item.printInvoice()

        print("No shipping cost for Digital Product.\n")

    elif isinstance(item, GiftCard):

        item.printInvoice()

        discount = item.calculateDiscount()

        print(f"Discount applied: {discount}\n")

if __name__ == "__main__":

    main()

```

Issues with the Code

- Doesn't Follow Single Responsibility Principle (SRP):**

The classes **PhysicalProduct**, **DigitalProduct**, and **GiftCard** contain both business logic and actions that should ideally be in separate classes. For example, printing invoices and calculating shipping costs or discounts are two separate responsibilities, but they're tightly coupled inside the same class.
- Product Type Checking in the Client Code:**

In the client code, we are performing checks like **instanceof PhysicalProduct**, **instanceof DigitalProduct**, and so on. This violates the **Open-Closed Principle (OCP)**, as adding a new product type would require modifying this client code. Ideally, we want to avoid such checks by delegating the operation logic to the product classes themselves.
- Lack of Flexibility:**

Adding a new product type in the future (say,

SubscriptionProduct) would require modifying the **Client Code**, which is not ideal for scalability. Every time a new product is added, the client code would have to be changed to account for new logic.

- **Tight Coupling:**

The product classes are tightly coupled to the specific operations (printing invoices, calculating shipping costs, and applying discounts). If you want to add more operations, you'd have to modify each product class, which can lead to a codebase that's hard to maintain.

The Solution

The issues in the previous code can be effectively addressed using the **Visitor Pattern**. The Visitor Pattern allows you to decouple operations from the objects on which they operate. It introduces two key components:

- **Element:**

This represents the objects on which operations (such as calculating shipping costs or generating invoices) will be performed. Each concrete element class (like **PhysicalProduct**, **DigitalProduct**, **GiftCard**) will implement an interface (such as **Item**) that defines an **accept()** method.

- **Visitor:**

The visitor is responsible for implementing the operations on the elements. It defines a **visit method** for each concrete element type, allowing new operations to be added without modifying the element classes themselves.

By using the **Visitor Pattern**, we can easily add new operations (like printing invoices or calculating shipping costs) without changing the classes of the products. We only need to implement a new visitor class.

Here's the updated code implementing the Visitor Pattern:

```
#include <iostream>
```

```
#include <vector>
```

```
#include <string>
```

```
using namespace std;
```

```
// ===== Element Interface =====
```

```
class Item {
```

```
public:
```

```
    virtual void accept(class ItemVisitor* visitor) = 0;
```

```
    virtual ~Item() {}
```

```
};
```

```
// ===== Concrete elements =====
```

```
class PhysicalProduct : public Item {
```

```
public:
```

```
    string name;
```

```
    double weight;
```

```
PhysicalProduct(string name, double weight) : name(name),  
weight(weight) {}
```

```
void accept(ItemVisitor* visitor) override;  
};
```

```
// ===== Concrete elements =====
```

```
class DigitalProduct : public Item {
```

```
public:
```

```
    string name;
```

```
    int downloadSizeInMB;
```

```
    DigitalProduct(string name, int downloadSizeInMB) : name(name),  
downloadSizeInMB(downloadSizeInMB) {}
```

```
    void accept(ItemVisitor* visitor) override;  
};
```

```
// ===== Concrete elements =====
```

```
class GiftCard : public Item {
```

```
public:
```

```
    string code;
```

```
double amount;
```

```
    GiftCard(string code, double amount) : code(code), amount(amount)
{}

```

```
    void accept(ItemVisitor* visitor) override;
};

```

```
// ===== Visitor Interface =====

```

```
class ItemVisitor {

```

```
public:

```

```
    virtual void visit(PhysicalProduct* item) = 0;

```

```
    virtual void visit(DigitalProduct* item) = 0;

```

```
    virtual void visit(GiftCard* item) = 0;

```

```
    virtual ~ItemVisitor() {}

```

```
};

```

```
// ===== Concrete Visitors =====

```

```
class InvoiceVisitor : public ItemVisitor {

```

```
public:

```

```
    void visit(PhysicalProduct* item) override {

```



```
        cout << "Invoice: " << item->name << " - Shipping to customer" << endl;
    }
}
```

```
void visit(DigitalProduct* item) override {
    cout << "Invoice: " << item->name << " - Email with download link" << endl;
}
}
```

```
void visit(GiftCard* item) override {
    cout << "Invoice: Gift Card - Code: " << item->code << endl;
}
};
```

```
// ===== Concrete Visitors =====
```

```
class ShippingCostVisitor : public ItemVisitor {
public:
    void visit(PhysicalProduct* item) override {
        cout << "Shipping cost for " << item->name << ": Rs. " << (item->weight
* 10) << endl;
    }
}
```

```
void visit(DigitalProduct* item) override {  
    cout << item->name << " is digital -- No shipping cost." << endl;  
}
```

```
void visit(GiftCard* item) override {  
    cout << "GiftCard delivery via email -- No shipping cost." << endl;  
}
```

```
};
```

```
// ===== Concrete element accept methods =====
```

```
void PhysicalProduct::accept(ItemVisitor* visitor) {  
    visitor->visit(this);  
}
```

```
void DigitalProduct::accept(ItemVisitor* visitor) {  
    visitor->visit(this);  
}
```

```
void GiftCard::accept(ItemVisitor* visitor) {  
    visitor->visit(this);  
}
```

```
// Client Code
```

```
int main() {
```

```
    vector<Item*> items;
```

```
    items.push_back(new PhysicalProduct("Shoes", 1.2));
```

```
    items.push_back(new DigitalProduct("Ebook", 100));
```

```
    items.push_back(new GiftCard("TUF500", 500));
```

```
    ItemVisitor* invoiceGenerator = new InvoiceVisitor();
```

```
    ItemVisitor* shippingCalculator = new ShippingCostVisitor();
```

```
    for (auto item : items) {
```

```
        item->accept(invoiceGenerator);
```

```
        item->accept(shippingCalculator);
```

```
        cout << endl;
```

```
    }
```

```
// Clean up
```

```
for (auto item : items) {
```

```
    delete item;
```

```
}
```

```
delete invoiceGenerator;
```

```
delete shippingCalculator;
```

```
return 0;
```

```
}
```

```
import java.util.*;
```

```
// ===== Element Interface =====
```

```
interface Item {
```

```
    void accept(ItemVisitor visitor);
```

```
}
```

```
// ===== Concrete elements =====
```

```
class PhysicalProduct implements Item {
```

```
    String name;
```

```
    double weight;
```

```
    public PhysicalProduct(String name, double weight) {
```

```
        this.name = name;
```

```
    this.weight = weight;
}
```

```
public void accept(ItemVisitor visitor) {
    visitor.visit(this);
}
}
```

```
// ===== Concrete elements =====
```

```
class DigitalProduct implements Item {
    String name;
    int downloadSizeInMB;

    public DigitalProduct(String name, int downloadSizeInMB) {
        this.name = name;
        this.downloadSizeInMB = downloadSizeInMB;
    }
}
```

```
public void accept(ItemVisitor visitor) {
    visitor.visit(this);
}
```

```
}
```

```
// ===== Concrete elements =====
```

```
class GiftCard implements Item {
```

```
    String code;
```

```
    double amount;
```

```
    public GiftCard(String code, double amount) {
```

```
        this.code = code;
```

```
        this.amount = amount;
```

```
    }
```

```
    public void accept(ItemVisitor visitor) {
```

```
        visitor.visit(this);
```

```
    }
```

```
}
```

```
// ===== Visitor Interface =====
```

```
interface ItemVisitor {
```

```
    void visit(PhysicalProduct item);
```

```
    void visit(DigitalProduct item);
```

```

    void visit(GiftCard item);
}

// ===== Concrete Visitors =====

class InvoiceVisitor implements ItemVisitor {

    public void visit(PhysicalProduct item) {

        System.out.println("Invoice: " + item.name + " - Shipping to
customer");

    }

    public void visit(DigitalProduct item) {

        System.out.println("Invoice: " + item.name + " - Email with download
link");

    }

    public void visit(GiftCard item) {

        System.out.println("Invoice: Gift Card - Code: " + item.code);

    }

}

// ===== Concrete Visitors =====

class ShippingCostVisitor implements ItemVisitor {

```

```

    public void visit(PhysicalProduct item) {

        System.out.println("Shipping cost for " + item.name + ": Rs. " +
(item.weight * 10));

    }

    public void visit(DigitalProduct item) {

        System.out.println(item.name + " is digital -- No shipping cost.");

    }

    public void visit(GiftCard item) {

        System.out.println("GiftCard delivery via email -- No shipping cost.");

    }

}

```

// Client Code

```

public class Main {

    public static void main(String[] args) {

        List<Item> items = new ArrayList<>();

        items.add(new PhysicalProduct("Shoes", 1.2));

        items.add(new DigitalProduct("Ebook", 100));

        items.add(new GiftCard("TUF500", 500));

    }

}

```



```

ItemVisitor invoiceGenerator = new InvoiceVisitor();

ItemVisitor shippingCalculator = new ShippingCostVisitor();


for (Item item : items) {

    item.accept(invoiceGenerator);

    item.accept(shippingCalculator);


    System.out.println("");
}
}
}

```

```

# ===== Element Interface =====

```

```

from abc import ABC, abstractmethod

```

```

class Item(ABC):

    @abstractmethod

    def accept(self, visitor):

        pass

```

```
# ===== Concrete elements =====
```

```
class PhysicalProduct(Item):
```

```
    def __init__(self, name, weight):
```

```
        self.name = name
```

```
        self.weight = weight
```

```
    def accept(self, visitor):
```

```
        visitor.visit(self)
```

```
# ===== Concrete elements =====
```

```
class DigitalProduct(Item):
```

```
    def __init__(self, name, downloadSizeInMB):
```

```
        self.name = name
```

```
        self.downloadSizeInMB = downloadSizeInMB
```

```
    def accept(self, visitor):
```

```
        visitor.visit(self)
```

```
# ===== Concrete elements =====
```

```
class GiftCard(Item):
```

```
    def __init__(self, code, amount):
```

```
self.code = code
```

```
self.amount = amount
```

```
def accept(self, visitor):
```

```
    visitor.visit(self)
```

```
# ===== Visitor Interface =====
```

```
class ItemVisitor(ABC):
```

```
    @abstractmethod
```

```
    def visit(self, item):
```

```
        pass
```

```
# ===== Concrete Visitors =====
```

```
class InvoiceVisitor(ItemVisitor):
```

```
    def visit(self, item):
```

```
        if isinstance(item, PhysicalProduct):
```

```
            print(f"Invoice: {item.name} - Shipping to customer")
```

```
        elif isinstance(item, DigitalProduct):
```

```
            print(f"Invoice: {item.name} - Email with download link")
```

```
        elif isinstance(item, GiftCard):
```

```
print(f"Invoice: Gift Card - Code: {item.code}")
```

```
# ===== Concrete Visitors =====
```

```
class ShippingCostVisitor(ItemVisitor):
```

```
    def visit(self, item):
```

```
        if isinstance(item, PhysicalProduct):
```

```
            print(f"Shipping cost for {item.name}: Rs. {item.weight * 10}")
```

```
        elif isinstance(item, DigitalProduct):
```

```
            print(f"{item.name} is digital -- No shipping cost.")
```

```
        elif isinstance(item, GiftCard):
```

```
            print("GiftCard delivery via email -- No shipping cost.")
```

```
# Client Code
```

```
def main():
```

```
    items = [
```

```
        PhysicalProduct("Shoes", 1.2),
```

```
        DigitalProduct("Ebook", 100),
```

```
        GiftCard("TUF500", 500)
```

```
    ]
```

```

invoiceGenerator = InvoiceVisitor()

shippingCalculator = ShippingCostVisitor()


for item in items:

    item.accept(invoiceGenerator)

    item.accept(shippingCalculator)

    print()


if __name__ == "__main__":

    main()

```

Let's now understand how the Visitor pattern solves the previously discussed issues.

How the Visitor Pattern Solves the Issues

Issue	How it is Solved
Doesn't Follow Single Responsibility	Each element class (PhysicalProduct , DigitalProduct , GiftCard) now only handles the representation of the product, delegating operations like printing invoices and calculating shipping costs to separate visitor classes.

Principle (SRP)

Product Type Checking in Client Code The client code (in **Main**) no longer needs to check the type of product with **instanceof**. Instead, it delegates the operation to the appropriate visitor via the **accept method**, which eliminates the need for explicit type checks.

Lack of Flexibility New operations (like printing invoices or calculating shipping costs) can now be added by simply creating a new visitor (e.g., **InvoiceVisitor** or **ShippingCostVisitor**). This adheres to the **Open-Closed Principle (OCP)**, as the product classes don't need to be modified to add new functionality.

Tight Coupling The coupling between operations (like invoice generation and shipping cost calculation) and product classes is eliminated. The operations are isolated in the visitor classes, which can be added or modified without affecting the product classes.

Double Dispatch

Double Dispatch is a technique used in object-oriented programming where a function call is dispatched to the method of an object, but the specific method is determined at runtime based on the type of two objects involved in the call.

It is commonly used in the **Visitor Pattern** to enable different actions depending on both the type of the visitor and the type of the element being visited. Here's how it works:

- **First Dispatch:** When an element (such as a product in the e-commerce example) accepts a visitor, the **accept()** method is called on the element, and the element dispatches the call to the visitor's **visit()** method.
- **Second Dispatch:** The **visit()** method in the visitor class is then responsible for determining the type of the element (like **PhysicalProduct**, **DigitalProduct**, or **GiftCard**). The second dispatch occurs as the method is chosen based on the type of both the visitor and the element.

Double Dispatch helps achieve flexibility and extendability in the system without introducing tight coupling between the classes involved. It ensures that the correct method is called at runtime based on the actual types of both the element and the visitor.

When to Use the Visitor Pattern

The Visitor Pattern is a powerful tool when dealing with complex object structures and performing operations that are independent of the object classes. It is especially useful in the following scenarios:

- **Complex Object Structure:**
If you have a complex object structure and need to perform unrelated operations on the elements, the Visitor Pattern helps by encapsulating these operations in separate visitor classes.
- **Adding Operations Without Modifying Element Classes:**
When you need to add new operations without modifying the

existing element classes, the Visitor Pattern allows you to introduce new functionality in a way that is decoupled from the existing object structure.

- **Distinct Types of Elements:**

If you have distinct types of elements that each require different logic for operations, the Visitor Pattern ensures that each element can interact with its own corresponding logic via the visitor.

- **Avoid Frequent Changes in Object Structure:**

The Visitor Pattern should be avoided if the object structure changes frequently, as it relies on the elements being stable in order to implement operations effectively.

Advantages and Disadvantages of Visitor Pattern

Pros

- **Follows OCP (Open-Closed Principle):**

The Visitor Pattern allows adding new operations without modifying the existing classes of the elements. This promotes extensibility without altering the core classes.

- **Clean Separation of Logic:**

The pattern helps keep the element classes free from operation-specific logic by centralizing all operations in separate visitor classes.

- **Easy to Add New Operations:**

When new operations need to be performed on elements, new visitor classes can be created without modifying the element classes, which makes the code more maintainable and flexible.

- **Helps Centralize Operations:**

The visitor centralizes operations related to specific tasks, making the code easier to understand, manage, and modify, especially when dealing with multiple different operations on the same object structure.

Cons

- **Adding New Elements Requires Modifying the Visitor Interface:**
When new element types are introduced, the visitor interface must be updated to accommodate the new type, requiring modifications to the existing visitor classes.
- **Can Be an Overkill for Simple Object Structures:**
For simple systems with few operations and elements, the Visitor Pattern might introduce unnecessary complexity, making the design harder to understand.
- **Double Dispatch Might Be Unintuitive for Some:**
The concept of double dispatch (where method calls depend on both the element type and visitor type) can be confusing and hard to follow for some developers.
- **Element and Visitor Classes are Tightly Coupled:**
While the pattern decouples operations from the elements, it still tightly couples the elements with the visitors, meaning any change in the visitor's interface could lead to changes in all element classes.

Real-Life Use Cases

The Visitor Pattern is widely used in various real-world applications due to its ability to add new operations without altering the object structures. Below are three examples of how the Visitor Pattern can be applied in real-life scenarios:

1. E-commerce Applications

In an e-commerce platform, different types of products (e.g., physical products, digital products, and gift cards) require different operations

(e.g., calculating shipping costs, applying discounts, and generating invoices). The Visitor Pattern can be used to centralize these operations.

How it Works:

Each product type (physical, digital, gift card) would accept a visitor (like **ShippingCostVisitor** or **InvoiceVisitor**) that performs the appropriate operation on the product. When a new operation (e.g., tax calculation) is needed, you can simply create a new visitor without modifying the product classes.

2. Compilers

In compilers, the Visitor Pattern is commonly used to traverse abstract syntax trees (ASTs). Different types of nodes in the tree (representing different parts of the code) require different operations, such as semantic analysis, optimization, or code generation.

How it Works:

The visitor can define specific actions (e.g., optimization or code generation) for different node types (e.g., loops, variables, conditionals) without modifying the node classes. As the compiler parses the code, the appropriate visitor is used to perform various operations on the AST nodes.

3. Abstract System Trees (Hierarchical Structures):

In complex systems that involve hierarchical data (e.g., file systems or organizational structures), the Visitor Pattern can be used to apply various

operations like searching, filtering, or reporting across the entire tree structure.

How it Works:

Each node in the tree represents an object in the hierarchy (e.g., a file or directory). The visitor performs operations such as calculating total storage space, generating reports, or searching for specific files. Adding new operations (like encryption or backup) can be easily done by creating new visitors without altering the tree structure.

Class Diagram

